

Algorithm Fundamentals

Principles of Program Development

1.1 Introduction Concepts

Understanding Algorithms

What is an Algorithm?

Definition

An algorithm is a finite, well-defined sequence of instructions or rules designed to solve a specific problem or perform a computation.

Think of it as a **recipe**: given specific ingredients (inputs), you follow exact steps to produce a dish (output) without any ambiguity.

Core Properties

- ▶ **Finiteness:** Must terminate after a finite number of steps.
- ▶ **Definiteness:** Each step must be rigorously and unambiguously defined.
- ▶ **I/O Mechanism:** Requires 0 or more inputs, and produces 1 or more outputs.
- ▶ **Effectiveness:** Operations must be basic enough to execute on a computer.

Problem Solving Stages



1. Analysis

Understand the problem requirements. Identify what the initial data (inputs) are and what the desired result (output) is.



2. Design

Draft the logic. This involves creating a flowchart or writing pseudocode to map out the algorithm's sequence step-by-step.



3. Implementation

Translate the logical design into a specific programming language (like Python, C++, or Java) for machine execution.

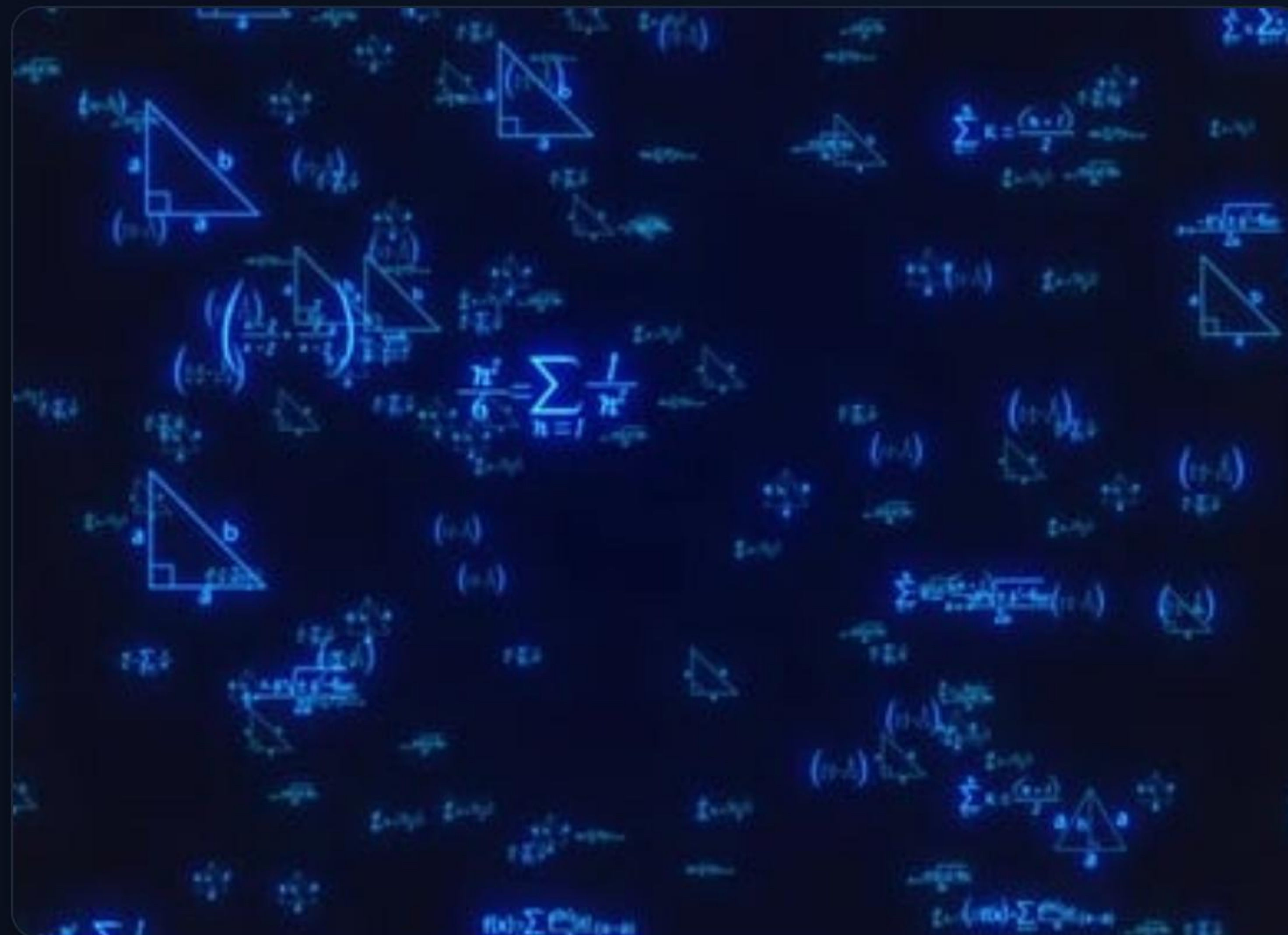
1.2 Algorithmic Data

Constants, Variables, and Expressions

Handling Data in Python

Algorithms manipulate data. **Constants** are values that never change during execution, while **Variables** act as containers that can hold changing values. We combine them using **Expressions**.

```
# Constant (By convention, all caps)  
PI = 3.14159  
  
# Variable (Can be overwritten)  
radius = 5  
  
# Expression (Combines operators/data)  
area = PI * (radius ** 2)  
  
# Output result  
print("Circle Area:", area)
```



1.3 Algorithm Representation

Control Structures & Flow

The Three Core Structures



Sequential

The default behavior. Instructions execute linearly, one exactly after the other, from top to bottom without skipping.



Alternative

Decision making. The algorithm checks a condition and forks its path, executing one block of code or another based on True/False.



Repetitive

Looping. A specific block of instructions is repeated continuously as long as a defining condition remains True.

Alternative Structure (if/else)

Using branching paths allows our programs to be dynamic. The code evaluates a boolean condition and chooses a specific execution route.

```
# Input
grade = float(input("Enter exam grade: "))

# Alternative decision flow
if grade ≥ 5.0:
    print("Status: Exam Passed!")
    print("Congratulations.")
else:
    print("Status: Failed.")
    print("Please study and try again.")
```



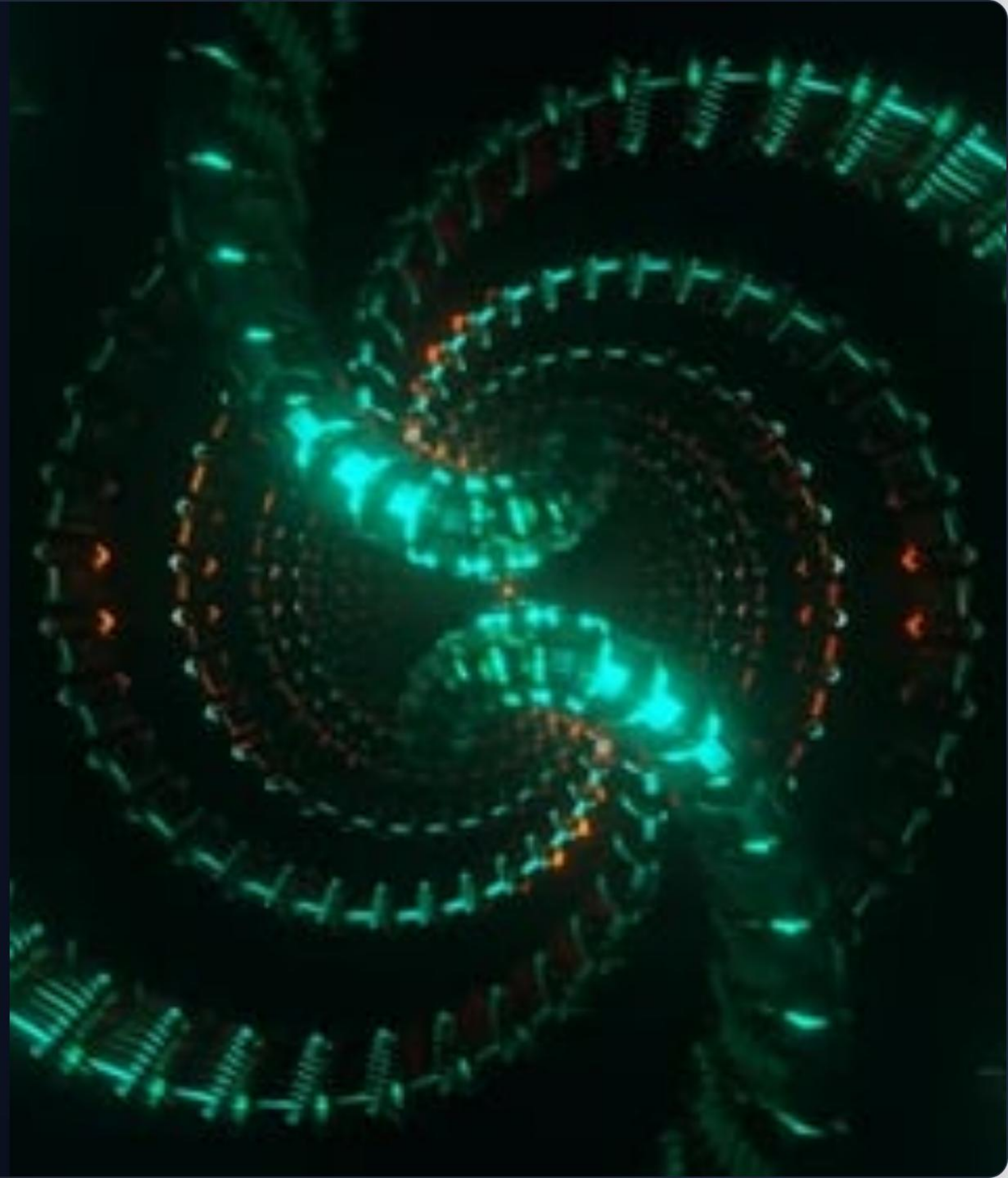
Repetitive Structure

Loops eliminate code redundancy by executing the same sequence of instructions multiple times.

```
# Calculate a factorial using a loop
target = 5
total = 1

for i in range(1, target + 1):
    total = total * i
    print("Step", i, "→", total)

print("\nFinal Factorial:", total)
```



End of Module 1

Questions regarding Data, Expressions, or Control Structures?

Continue practicing with your IDE

Image Sources



https://elements-resized.envatousercontent.com/elements-video-cover-images/0084181b-f79a-409e-9941-4f8fc8c2ba2b/video_preview/video_preview_0002.jpg?w=500&cf_fit=cover&q=85&format=auto&s=1c0f44b49b57b18bd52137ebcc3a4071b069162147e224f157485594731cc4d5

Source: elements.envato.com



https://images.pond5.com/neon-arrows-diverging-choices-path-footage-308708387_icon1.jpeg

Source: www.pond5.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/41e970c9-f1a0-4faf-a89a-a64719d4d9fb/video_preview/video_preview_0000.jpg?w=500&cf_fit=cover&q=85&format=auto&s=3f2ecf546f8f202ac2431ad983a26da3185c3f742d69b286ecc3b2305972fab0

Source: elements.envato.com